

Exercise Session 3

Theory

- ROS parameter
- ROS action
- ROS server

Exercise

Overview

This homework is designed to give you hands-on experience with implementing a path planning algorithm using the Robot Operating System (ROS). You will work with the Rapidly-exploring Random Tree (RRT) algorithm, integrate it into a ROS action server, and observe the planned paths in a simulated environment.

Before starting the tasks, download the exercise files from the course website and move them into the `/src` folder of a newly created colcon workspace.

Task 1 - Set up the mission through ROS parameters

In this task, you will set the mission, i.e., the start and end position of the planning, through the parameter server. We only plan in 2D, therefore both start and goal should contain two coordinates, `x` and `y`. After the mission information is put on the parameter server as parameters, you will need to get them from the action client and send it to the action server.

In detail, you are required to:

- Create a `params.yaml` file under the folder `/config`.
- In the `yaml` file, define four parameters: `start.x`, `start.y`, `end.x`, `end.y`, which define the scope of our mission. Set start at `[0, 0]` and end at `[15,10]`. **Hint:** Think which node actually *needs* access to the mission parameters.
- In order to make the above parameters accessible to the node, modify the launch file accordingly.
- In the action client node (`planning_action_client.cpp`), declare and get the parameters from the parameter server.
- Format the mission definition goal from the parameter server appropriately according to the action definition (see `PlanPath.action`). The mission should correspond to the Goal section in the action. **Note:** Please use `map` as `frame_id` throughout this exercise.
- Send the Goal to the action server.

Hint: At this point you can build the whole package and test your implementation by launching `planning_action.launch.py` and checking whether the parameters exist on the parameters server via command-line, such as `ros2 param list`. A visualization node is also implemented for you, so if everything is correct, you should see the start and end position visualized in RViz as shown in Fig. 1.

Note: If you don't see the start and end in rviz, it's probably because the rviz started too late,

you can increase the sleep duration in

```
rclcpp::sleep_for(std::chrono::milliseconds(500));
```

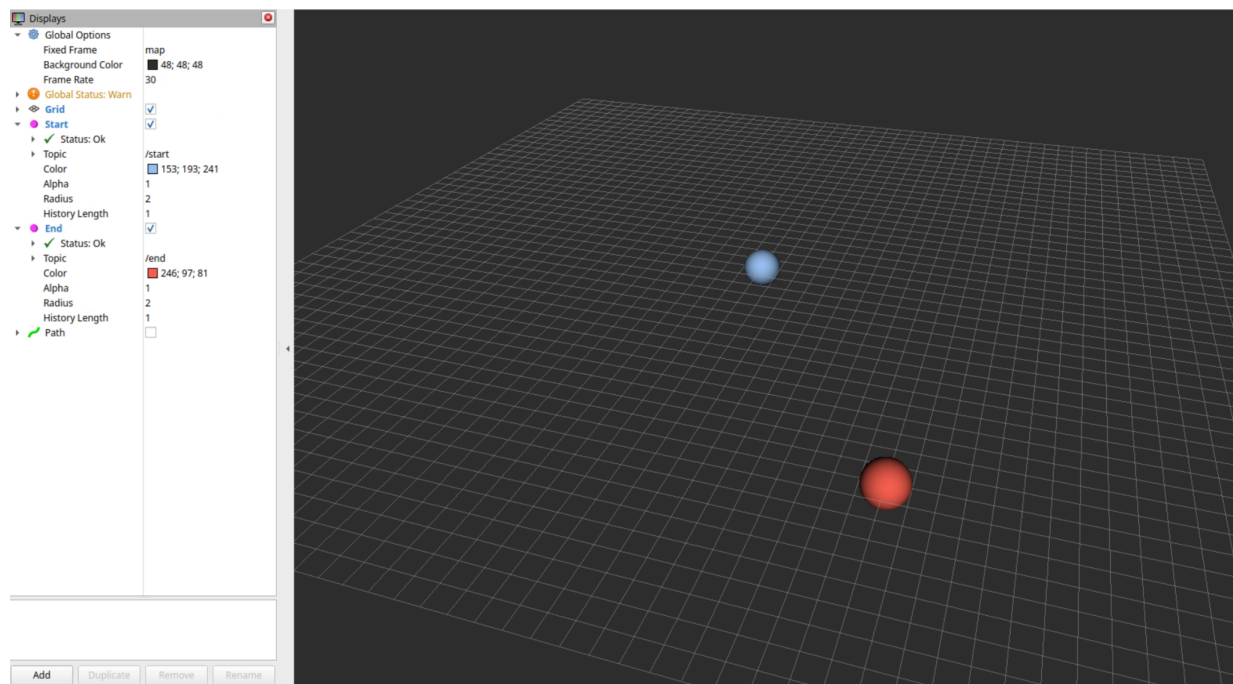


Fig. 1. You should see the start in blue and end in red in RViz if the goal is sent to the action server correctly.

Task 2 - Implement the action server

An advantage of action compared to service is that the action server can inform the action client about the incremental progress of a goal. This is possible via the feedback defined in the *.action* file. The action server can continuously send feedback to the action client in its main execution callback. In this task, you will complete the *executeCB(...)* method defined in *planning_action_server.cpp*. The action server will first plan the path and follow the path step by step while keeping sending feedback to the client at each step. Specifically you are asked to:

- Convert the received goal to appropriate type for the provided RRT planner.
- Complete the provided simple RRT planner so it can plan a path from the start to the end. For this, you only need to fill *plan()* in *rrt_planner.h*. Necessary helper functions are already implemented, you just need to implement the simple algorithmic logic:
 - Sample a random point.
 - Connect it to the nearest point.
 - Return the path if reached the goal or the *max_iteraton* is reached, otherwise keep sampling.
- Use the completed RRT planner to plan the path.
- For each step of the planned path, send a feedback to the client that includes the executed (followed) path until this step, and the completed percentage of the path. Please refer to *PlanPath.action* for the required datatype. **Hint:** It is sufficient to

calculate the percentage via $\text{executed step} / \text{total step}$ instead of the actual path length.

- Send the final result after the entire path is executed.
- If no path is found, handle it appropriately using `abort(...)`. **Note:** Although this should not happen for this exercise, since the planning task is expected to be succeeded definitely, you should handle this for completeness.

Hint: At this point, if the implementation is correct, you should see the path being executed step by step in the RViz as shown in Fig. 2., after building and launching the launch file.

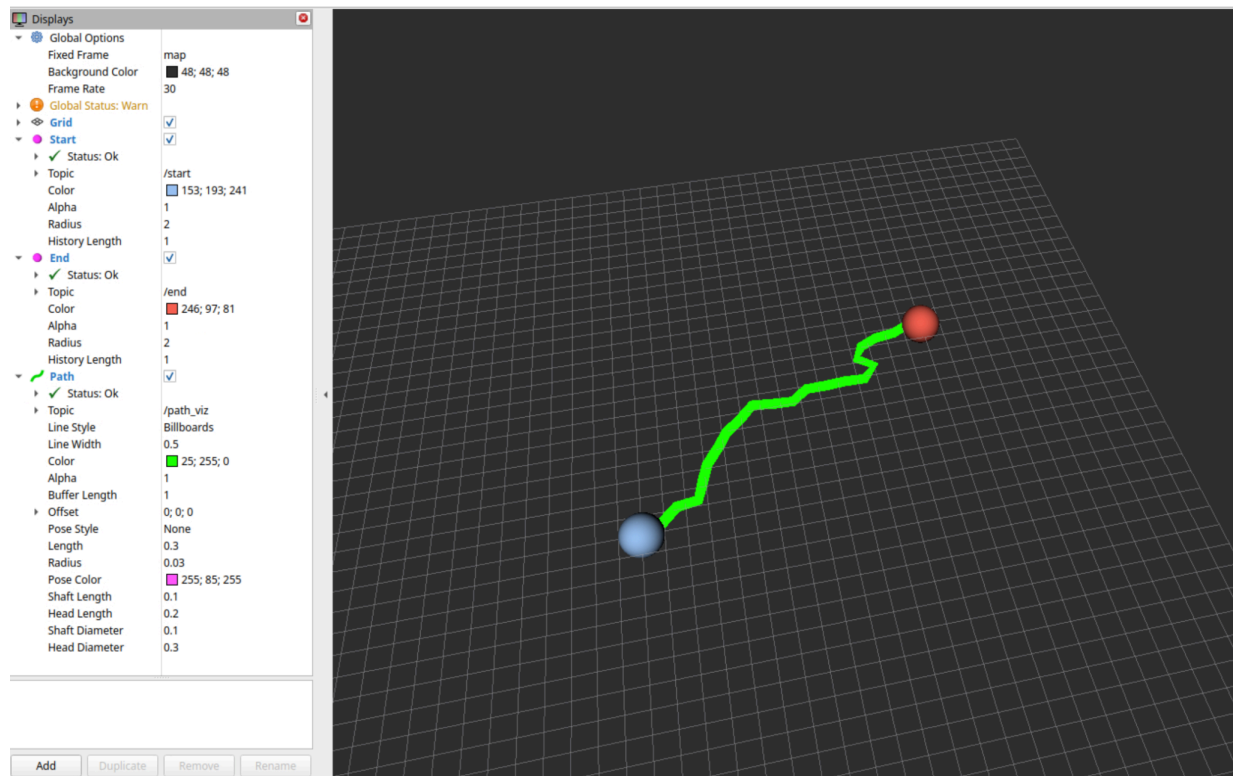


Fig. 2. Planned path visualized in Rviz.

Task 3 - Complete the action client

The action server is now ready. To establish a proper action server client communication the client still needs to implement some functions to receive information from the server. In this task, you will complete the implementation of the action client by implementing the various callback functions to get continuous feedback, active and done notifications. In detail you need to:

- Implement `goalResponseCb(...)` to print accepted/rejected messages (accepted == goal is active for this exercise) using the following two messages: “Goal accepted by server, waiting for result...”, or, “Goal rejected by server.”
- Complete the `feedbackCb(...)` in the action client. Print out the current percentage of the completed path in the format: “Got Feedback: XXX.XX percent complete”, where XXX.XX refers to the calculated percentage (e.g. 34.78) .
- Complete the `doneCb(...)` in the action client. You need to:

- Print out the goal state in the following format (success or not): “Finished in state [SUCCEEDED]” or “Finished in state [FAILED]”.
- Publish the status on the topic *planner_status*. The message type should be a boolean with true indicating succeeded and false otherwise. **Hint:** For this you will need to create another publisher.

Now everything is ready, you can test if your code is implemented correctly by examining if the paths are visualized correctly in the RViz, if feedbacks are printed properly on the console, and if the *planner_status* correctly receives the message.

Submission

Submit all source code in the src folder of your submission (copy the whole package folder). Rename your submission folder to “exercise3_YOURSTUDENTID” (e.g. exercise3_12345678) and zip it. The zipped folder should have the same name as the submission folder.

Tip: Check that all the print messages are exactly as given in the exercise, otherwise you might lose points with the autograder.